PES Board — main.cpp

Differential-Drive Line-Following Robot with Parcel Pickup/Drop
STM32 Nucleo F446RE · Mbed OS · PlatformIO

```cpp
/**
 * @file  main.cpp
 * @brief PES Board     differential-drive line-following robot with parcel pickup/drop.
 *
 * State machine:
 *   INITIAL     FORWARD     COLOR_SCAN     HANDLE_PARCEL     INITIAL
 *   (after 4 pickup+drop cycles: INITIAL     SLEEP)
 *
 * Hardware: STM32 Nucleo F446RE    Mbed OS    PlatformIO / gcc
 */

#include "mbed.h"
#include "PESBoardPinMap.h"

// Drivers
#include "DCMotor.h"
#include "DebounceIn.h"
#include "FastPWM.h"
#include <Eigen/Dense>
#include "SensorBar.h"
#include "ColorSensor.h"
#include "Servo.h"
//#include "IIRFilter.h"

#define M_PIf 3.14159265358979323846f

// ---------------------------------------------------------------------------
// Global flags     toggled by the blue button ISR
// ---------------------------------------------------------------------------
bool do_execute_main_task = false; // true while the robot should run
bool do_reset_all_once    = false; // triggers a one-shot reset when task stops

// Blue button: falling edge calls the toggle function
DebounceIn user_button(BUTTON1);
void toggle_do_execute_main_fcn();

// ---------------------------------------------------------------------------
// Cross-detection background thread (4 ms polling)
// ---------------------------------------------------------------------------
volatile bool pickup_cross    = false; // full-bar pattern seen      pickup crossing
volatile bool drop_cross      = false; // center-only pattern seen     drop crossing
volatile bool color_detected  = false; // color identified this crossing; suppresses re-trigger
volatile int  cross_bar_count = 0;     // skips the very first bar hit so the robot gets on-track
volatile int  cross_lock_ticks = 0;    // cooldown counter (ticks) to avoid double-detection

SensorBar* sensor_bar_ptr = nullptr;   // set before thread start; read by detect_cross_thread
void detect_cross_thread();

int main()
{
    user_button.fall(&toggle_do_execute_main_fcn);

    // ---------------------------------------------------------------------------
    // Timing     fixed 20 ms (50 Hz) control loop
    // ---------------------------------------------------------------------------
    const int main_task_period_ms = 20;
    Timer     main_task_timer;
    main_task_timer.start();

    // ---------------------------------------------------------------------------
    // Peripherals
    // ---------------------------------------------------------------------------
```

```cpp
    DigitalOut user_led(LED1);
    DigitalOut enable_motors(PB_ENABLE_DCMOTORS); // power rail for all DC motors

    // ------------------------------------------------------------------------
    // DC Motors
    // M1/M2     drive wheels (motion planner disabled; direct velocity commands)
    // M3         crane arm (motion planner enabled for smooth position control)
    // ------------------------------------------------------------------------
    const float voltage_max = 12.0f;          // battery pack voltage (6.0f for single pack)
    const float gear_ratio  = 100.00f;
    const float kn          = 140.0f / 12.0f; // motor speed constant [rpm/V]

    DCMotor motor_M1(PB_PWM_M1, PB_ENC_A_M1, PB_ENC_B_M1, gear_ratio, kn, voltage_max);
    DCMotor motor_M2(PB_PWM_M2, PB_ENC_A_M2, PB_ENC_B_M2, gear_ratio, kn, voltage_max);
    DCMotor motor_M3(PB_PWM_M3, PB_ENC_A_M3, PB_ENC_B_M3, gear_ratio, kn, voltage_max);

    motor_M3.enableMotionPlanner();
    motor_M3.setMaxVelocity(motor_M3.getMaxPhysicalVelocity() * 0.75f); // cap for safety

    // ------------------------------------------------------------------------
    // Servos (pulse widths from calibration)
    // D0     arm, D1     gripper
    // ------------------------------------------------------------------------
    Servo servo_D0(PB_D0, 0.03f, 0.125f);
    Servo servo_D1(PB_D1, 0.03f, 0.1f);

    servo_D0.setMaxAcceleration(0.3f); // default 1.0e6f     slowed for smooth motion
    servo_D1.setMaxAcceleration(0.3f);

    const float servo_D0_home = 0.8f; // retracted / home pulse width
    const float servo_D1_home = 0.6f;

    // Used only in CALIBRATE_SERVO
    float servo_input   = 0.0f;
    int   servo_counter = 0;
    const int loops_per_seconds = static_cast<int>(ceilf(1.0f / (0.001f *
        ↪ static_cast<float>(main_task_period_ms))));

    // ------------------------------------------------------------------------
    // Differential-Drive Kinematics (Eigen)
    // Cwheel2robot maps robot-frame [v,   ]     wheel speeds.
    // Inverted in FORWARD to get per-wheel commands from desired robot motion.
    // ------------------------------------------------------------------------
    const float r_wheel           = 0.03f / 2.0f; // wheel radius [m]
    const float b_wheel           = 0.1408f;       // wheelbase center-to-center [m]
    const float max_speed_percentage{1.0f};        // forward speed scale factor (0  1  )

    Eigen::Matrix2f Cwheel2robot;
    Cwheel2robot <<  r_wheel / 2.0f,    r_wheel / 2.0f,
                     r_wheel / b_wheel, -r_wheel / b_wheel;

    // ------------------------------------------------------------------------
    // Sensor Bar (8-LED optical line sensor)
    // ------------------------------------------------------------------------
    const float bar_dist = 0.18f; // sensor-bar distance from wheel axis [m]
    SensorBar   sensor_bar(PB_9, PB_8, bar_dist);

    float       angle{0.0f};
    const float default_turning_angle(1.0f); // fallback when no line detected (spin to
        ↪ re-acquire)

    // ------------------------------------------------------------------------
    // Line-Following PD Controller
    // ------------------------------------------------------------------------
    const float Kp{12.5f};
    const float Kd{0.25f};
    const float dt = main_task_period_ms * 1e-3f; // control period [s]
    float       prev_error{0.0f};
    //IIRFilter angle_filter;
    //angle_filter.lowPass1Init(5.0f, dt); // low-pass 5 Hz to reduce sensor noise

    // Maximum tangential wheel speed [m/s], used to scale the forward velocity command
    const float wheel_vel_max = 2.0f * M_PIf * motor_M2.getMaxPhysicalVelocity();

    // ------------------------------------------------------------------------
```

```
136     // Color Sensor
137     // ------------------------------------------------------------------------
138     ColorSensor color_sensor(PB_3);
139 
140     int  current_color_run{0}; // color ID of the parcel currently being handled (0 = none)
141     bool color_done[9] = {};   // which colors have already been picked up
142     int  pickups_done  = 0;    // completed pickup+drop cycles
143 
144     // Color IDs returned by ColorSensor::getColor()
145     enum Color {
146         UNKNOWN = 0,
147         BLACK   = 1,
148         WHITE   = 2,
149         RED     = 3,
150         YELLOW  = 4,
151         GREEN   = 5,
152         CYAN    = 6,
153         BLUE    = 7,
154         MAGENTA = 8
155     };
156 
157     //int sleep_count = 0;
158 
159     // ------------------------------------------------------------------------
160     // Robot State Machine
161     // ------------------------------------------------------------------------
162     enum RobotState {
163         INITIAL,         // home servos, enable motors, choose next state
164         SLEEP,           // all tasks done     motors off indefinitely
165         FORWARD,         // line following with PD controller
166         COLOR_SCAN,      // stopped at crossing, identify parcel color
167         HANDLE_PARCEL,   // execute pickup or drop sequence
168         CALIBRATE_COLOR, // debug: stream raw RGBC readings
169         CALIBRATE_SERVO  // debug: sweep servo pulse width
170     } robot_state = RobotState::INITIAL;
171 
172     // Hand the sensor_bar pointer to the background thread before starting it
173     sensor_bar_ptr = &sensor_bar;
174     Thread cross_thread;
175     cross_thread.start(detect_cross_thread);
176 
177     bool is_pickup = true; // true = next HANDLE_PARCEL is a pickup, false = drop
178 
179     // ------------------------------------------------------------------------
180     // Crane Parameters     per color, per operation
181     //   m3_offset   : signed rotation added to M3 current position (extend/retract)
182     //   srv_out_d0  : arm servo pulse width during operation
183     //   srv_out_d1  : gripper servo pulse width during operation
184     // ------------------------------------------------------------------------
185     struct ParcelParams { float m3_offset, srv_out_d0, srv_out_d1; };
186 
187     const ParcelParams pickup_params[9] = {
188         {},                          // UNKNOWN
189         {},                          // BLACK
190         {},                          // WHITE
191         { 1.75f, 0.95f, 0.15f },     // RED         down right
192         { 1.75f, 0.95f, 0.2f  },     // YELLOW      up right
193         {-0.55f, 0.85f, 0.1f  },     // GREEN       down left
194         {},                          // CYAN
195         {-0.55f, 1.0f,  0.15f },     // BLUE        up left
196         {},                          // MAGENTA
197     };
198 
199     const ParcelParams drop_params[9] = {
200         {},                          // UNKNOWN
201         {},                          // BLACK
202         {},                          // WHITE
203         { 1.75f, 0.95f, 0.1f  },     // RED
204         { 1.75f, 0.9f,  0.2f  },     // YELLOW
205         {-0.55f, 0.85f, 0.1f  },     // GREEN
206         {},                          // CYAN
207         {-0.6f,  1.1f,  0.17f },     // BLUE
208         {},                          // MAGENTA
209     };
210 
```

```
    // ========================================================================
    // Main loop     executes every main_task_period_ms milliseconds
    // ========================================================================
    while (true) {
        main_task_timer.reset();

        if (do_execute_main_task) {
            // ------------------------------------------------------------
            // Active: blue button has been pressed     run the state machine
            // ------------------------------------------------------------
            switch (robot_state) {

                // ------------------------------------------------------------
                // INITIAL: enable motors, home servos, then start driving.
                // Waits until both servos reach home position before advancing.
                // ------------------------------------------------------------
                case RobotState::INITIAL: {
                    enable_motors   = 1;
                    pickup_cross    = false;
                    drop_cross      = false;
                    cross_bar_count = 0;

                    if (!servo_D0.isEnabled()) servo_D0.enable();
                    if (!servo_D1.isEnabled()) servo_D1.enable();
                    servo_D0.setPulseWidth(servo_D0_home);
                    servo_D1.setPulseWidth(servo_D1_home);

                    if (fabsf(servo_D0.getPulseWidth() - servo_D0_home) < 0.02f &&
                        fabsf(servo_D1.getPulseWidth() - servo_D1_home) < 0.02f) {
                        robot_state = (pickups_done >= 4) ? RobotState::SLEEP :
                            ↪ RobotState::FORWARD;
                    }
                    break;
                }

                // ------------------------------------------------------------
                // SLEEP: mission complete     keep motors off indefinitely
                // ------------------------------------------------------------
                case RobotState::SLEEP: {
                    enable_motors = 0;
                    break;
                }

                // ------------------------------------------------------------
                // FORWARD: follow the black line using PD control on sensor angle
                // ------------------------------------------------------------
                case RobotState::FORWARD: {
                    if (sensor_bar.isAnyLedActive()) {
                        uint8_t raw = sensor_bar.getRaw();

                        // Count active LEDs to detect a wide crossing (  4   LEDs lit)
                        int active_leds = 0;
                        for (int i = 0; i < 8; i++) {
                            if (raw & (1 << i)) active_leds++;
                        }

                        // During a drop run a wide bar means we're at the crossing;
                        // bias angle slightly left to align with the drop position.
                        if (is_pickup == false && active_leds >= 4) {
                            angle = -0.15f;
                        } else {
                            angle = sensor_bar.getAvgAngleRad();
                        }

                        // Cross detected by background thread     stop and scan color
                        if ((pickup_cross || drop_cross) && !color_detected) {
                            prev_error  = 0.0f;
                            robot_state = RobotState::COLOR_SCAN;
                            break;
                        }
                    } else {
                        //angle = 0.0f; // straight ahead when no line
                        angle = default_turning_angle; // no line     spin to re-acquire
                    }

```

```
285                     // PD controller: error = measured angle, output = angular velocity
286                     color_detected = false;
287                     const float angle_threshold             = -0.3;
288                     const float derivative_threshold        = Kd * angle_threshold; // Kd * -0.3
289                     const float derivative_correction_factor = 16.0f;
290                     float derivative = (angle - prev_error) / dt;
291                     float omega      = -(Kp * angle + Kd * derivative);
292                     prev_error       = angle;
293
294                     //if (Kd * derivative < derivative_threshold) {
295                     //    omega *= derivative_correction_factor;
296                     //    //printf("omega: %f, angle: %f, Kd, %f, derivative %f\n", omega,
                            ↪ angle, Kd, derivative);
297                     //}
298
299                     // slow down on sharp turns, speed up on straights
300                     //float speed_factor = 1.0f - fabsf(angle) / default_turning_angle;
301                     //speed_factor = (speed_factor < 0.3f) ? 0.3f : speed_factor;
302
303                     // Convert desired robot velocities to individual wheel speeds [rot/s]
304                     Eigen::Vector2f robot_coord = {max_speed_percentage * wheel_vel_max *
                            ↪ r_wheel, omega};
305                     //Eigen::Vector2f robot_coord = {speed_factor * max_speed_percentage *
                            ↪ wheel_vel_max * r_wheel, omega};
306                     Eigen::Vector2f wheel_speed = Cwheel2robot.inverse() * robot_coord;
307
308                     // M2 is mounted mirrored on the chassis     negate its velocity
309                     motor_M1.setVelocity( wheel_speed(0) / (2.0f * M_PIf));
310                     motor_M2.setVelocity(-wheel_speed(1) / (2.0f * M_PIf));
311
312                     //printf("angle: %f | omega: %f\n | lin_speed: %f\n", angle, omega,
                            ↪ max_speed_percentage * wheel_vel_max * r_wheel);
313                     break;
314                 }
315
316                 // ------------------------------------------------------------
317                 // COLOR_SCAN: robot stopped at crossing     identify parcel color.
318                 // Decides whether to pickup, drop, or ignore this crossing.
319                 // ------------------------------------------------------------
320                 case RobotState::COLOR_SCAN: {
321                     motor_M1.setVelocity(0.0f);
322                     motor_M2.setVelocity(0.0f);
323                     color_sensor.switchLed(ON);
324
325                     int detected = color_sensor.getColor();
326
327                     if (detected == YELLOW || detected == GREEN ||
328                         detected == RED    || detected == BLUE) {
329                         color_sensor.switchLed(OFF);
330
331                         if (!color_done[detected] && pickup_cross && current_color_run == 0) {
332                             // New color at a pickup cross     begin pickup sequence
333                             color_done[detected] = true;
334                             current_color_run    = detected;
335                             color_detected       = true;
336                             //printf("Color: %s\n",
                                    ↪ ColorSensor::getColorString(current_color_run));
337                             is_pickup   = true;
338                             robot_state = RobotState::HANDLE_PARCEL;
339                             //robot_state = RobotState::FORWARD;
340                         } else if (current_color_run == detected) {
341                             // Same color as carried parcel     this is the drop-off spot
342                             drop_cross   = false;
343                             pickup_cross = false;
344                             is_pickup    = false;
345                             robot_state  = RobotState::HANDLE_PARCEL;
346                             //robot_state = RobotState::FORWARD;
347                         } else {
348                             // Wrong color or already handled     skip and keep driving
349                             drop_cross   = false;
350                             pickup_cross = false;
351                             robot_state  = RobotState::FORWARD;
352                         }
353                     }
354                     break;
```

```
355                 }
356
357                 // ------------------------------------------------------------
358                 // HANDLE_PARCEL: sequenced crane movement for pickup or drop.
359                 // Each step polls a tolerance condition before advancing to the next.
360                 //
361                 //   1. MOVE_OUT_SRV1      position gripper servo (D1)
362                 //   2. MOVE_OUT_SRV0      position arm servo (D0)
363                 //   3. MOVE_OUT_M3        extend crane (M3)
364                 //   4. MOVE_BACK_M3       retract crane
365                 //   5. MOVE_BACK_SRVS     return both servos to home
366                 // ------------------------------------------------------------
367                 case RobotState::HANDLE_PARCEL: {
368                     motor_M1.setVelocity(0.0f);
369                     motor_M2.setVelocity(0.0f);
370                     user_led = 1;
371
372                     static enum class HandleStep {
373                         MOVE_OUT_SRV1,
374                         MOVE_OUT_SRV0,
375                         MOVE_OUT_M3,
376                         MOVE_BACK_M3,
377                         MOVE_BACK_SRVS,
378                     } step = HandleStep::MOVE_OUT_SRV1;
379                     static float m3_target = 0.0f;
380
381                     // Select crane parameters for the current color and operation
382                     const ParcelParams& p = is_pickup ? pickup_params[current_color_run]
383                                                       : drop_params[current_color_run];
384
385                     switch (step) {
386                         case HandleStep::MOVE_OUT_SRV1: {
387                             if (!servo_D1.isEnabled()) servo_D1.enable();
388                             servo_D1.setPulseWidth(p.srv_out_d1);
389                             if (fabsf(servo_D1.getPulseWidth() - p.srv_out_d1) < 0.02f)
390                                 step = HandleStep::MOVE_OUT_SRV0;
391                             break;
392                         }
393                         case HandleStep::MOVE_OUT_SRV0: {
394                             if (!servo_D0.isEnabled()) servo_D0.enable();
395                             servo_D0.setPulseWidth(p.srv_out_d0);
396                             if (fabsf(servo_D0.getPulseWidth() - p.srv_out_d0) < 0.02f) {
397                                 m3_target = motor_M3.getRotation() + p.m3_offset;
398                                 step      = HandleStep::MOVE_OUT_M3;
399                             }
400                             break;
401                         }
402                         case HandleStep::MOVE_OUT_M3: {
403                             motor_M3.setRotation(m3_target);
404                             if (fabsf(motor_M3.getRotation() - m3_target) < 0.05f) {
405                                 m3_target = motor_M3.getRotation() - p.m3_offset;
406                                 step      = HandleStep::MOVE_BACK_M3;
407                             }
408                             break;
409                         }
410                         case HandleStep::MOVE_BACK_M3: {
411                             motor_M3.setRotation(m3_target);
412                             if (fabsf(motor_M3.getRotation() - m3_target) < 0.05f)
413                                 step = HandleStep::MOVE_BACK_SRVS;
414                             break;
415                         }
416                         case HandleStep::MOVE_BACK_SRVS: {
417                             servo_D0.setPulseWidth(servo_D0_home);
418                             servo_D1.setPulseWidth(servo_D1_home);
419                             if (fabsf(servo_D0.getPulseWidth() - servo_D0_home) < 0.02f &&
420                                 fabsf(servo_D1.getPulseWidth() - servo_D1_home) < 0.02f) {
421                                 step         = HandleStep::MOVE_OUT_SRV1;
422                                 pickup_cross = false;
423                                 drop_cross   = false;
424
425                                 // Lock out cross detection for 125 ticks (    0.5 s at 4 ms
                                        ↪ polling)
426                                 // to prevent immediately re-triggering on the same crossing.
427                                 cross_lock_ticks = 125;
428                                 pickups_done++;
```

```
                                if (!is_pickup) current_color_run = 0; // clear carried color after drop
                                robot_state = RobotState::INITIAL;
                            }
                            break;
                        }
                    }
                    break;
                }

                // ------------------------------------------------------------
                // CALIBRATE_COLOR: stream raw RGBC values for sensor tuning
                // ------------------------------------------------------------
                case RobotState::CALIBRATE_COLOR: {
                    color_sensor.switchLed(ON);
                    const float* colors = color_sensor.readColor();
                    printf("R: %.2f\t G: %.2f\t B: %.2f\t C: %.2f\n",
                           colors[0], colors[1], colors[2], colors[3]);
                    break;
                }

                // ------------------------------------------------------------
                // CALIBRATE_SERVO: slowly sweep D1 pulse width and print it
                // ------------------------------------------------------------
                case RobotState::CALIBRATE_SERVO: {
                    if (!servo_D0.isEnabled()) servo_D0.enable();
                    if (!servo_D1.isEnabled()) servo_D1.enable();

                    //servo_D0.setPulseWidth(servo_input);
                    servo_D1.setPulseWidth(servo_input);

                    // Increment pulse width by 0.005 once per second
                    if ((servo_input < 1.0f) &&
                        (servo_counter % loops_per_seconds == 0) &&
                        (servo_counter != 0))
                        servo_input += 0.005f;
                    servo_counter++;

                    printf("Pulse width: %f\n", servo_input);
                    break;
                }

                default:
                    break;
            }

        } else {
            // ------------------------------------------------------------------
            // Inactive: blue button not pressed.
            // The reset block runs exactly once when execution is stopped.
            // ------------------------------------------------------------------
            if (do_reset_all_once) {
                do_reset_all_once = false;

                // Disable servos and reset calibration state
                servo_D0.disable();
                servo_D1.disable();
                servo_input   = 0.0f;
                servo_counter = 0;
            }
        }

        // toggling the user led
        //user_led = !user_led;

        // Sleep for the remainder of the 20 ms period (non-blocking)
        int main_task_elapsed_time_ms =
            duration_cast<milliseconds>(main_task_timer.elapsed_time()).count();
        if (main_task_period_ms - main_task_elapsed_time_ms < 0)
            printf("Warning: Main task took longer than main_task_period_ms\n");
        else
            thread_sleep_for(main_task_period_ms - main_task_elapsed_time_ms);
    }
}
```

```
// ------------------------------------------------------------------------
// Button ISR     toggles task execution on falling edge (button press)
// ------------------------------------------------------------------------
void toggle_do_execute_main_fcn()
{
    do_execute_main_task = !do_execute_main_task;
    if (do_execute_main_task)
        do_reset_all_once = true; // schedule one-shot reset on re-enable
}

// ------------------------------------------------------------------------
// Background thread     polls the sensor bar every 4 ms for crossing detection.
//
// Pattern recognition:
//   raw == 0xff  (all 8 LEDs active)        full bar      sets pickup_cross (rising edge only)
//   raw == 0x3c  (center 4 LEDs active)     center        sets drop_cross   (rising edge only)
//
// cross_lock_ticks: cooldown after HANDLE_PARCEL to prevent double-triggering.
// cross_bar_count:  skips the first full-bar hit so the robot can get onto the track.
// ------------------------------------------------------------------------
void detect_cross_thread()
{
    bool prev_full_line  = false;
    bool prev_cross_line = false;

    while (true) {
        uint8_t raw = sensor_bar_ptr->getRaw();
        bool full_line  = (raw == 0xff);
        bool cross_line = (raw == 0x3c);

        if (cross_lock_ticks > 0) cross_lock_ticks--;

        if (cross_lock_ticks == 0) {
            // Rising edge on center-only pattern     drop crossing
            if (!drop_cross && cross_line && !prev_cross_line)
                drop_cross = true;

            // Rising edge on full bar     pickup crossing (skip the very first hit)
            if (!pickup_cross && full_line && !prev_full_line) {
                // cross_bar_count skips the first cross to get on the track
                if (cross_bar_count == 0) cross_bar_count++;
                else pickup_cross = true;
            }
        }

        prev_full_line  = full_line;
        prev_cross_line = cross_line;
        ThisThread::sleep_for(4ms);
    }
}
```